>>> 50 Node. js Question

**Node.js Basics:**

1. What is Node.js, and how does it differ from client-side JavaScript?
2. Explain the non-blocking, event-driven architecture of Node.js.
3. How can you run a simple JavaScript file using Node.js?
4. What is the Node.js package manager, and why is it important?
5. Describe the CommonJS module system and how it's used in Node.js.

**Event Loop and Asynchronous Programming:**

6. What is the event loop in Node.js, and how does it work?
7. How does Node.js handle asynchronous operations?
8. Explain the callback pattern in Node.js and potential issues with callback hell.
9. What is the purpose of promises and async/await in handling asynchronous code in Node.js?
10. Describe the role of the "timers" module in scheduling asynchronous tasks.

**Node.js Modules:**

11. How do you create and use modules in Node.js?
12. What is the difference between built-in, core, and third-party modules in Node.js?
13. How can you load and use a module from an external file in Node.js?
14. Explain the significance of the `module.exports` and `exports` objects in Node.js.
15. What is the "require" function, and how does it work to load modules?

**File System Operations:**

16. How can you perform file I/O operations in Node.js?
17. Explain the difference between synchronous and asynchronous file operations in Node.js.
18. What is the "fs" module in Node.js, and how is it used for file system operations?
19. How do you read and write files using the "fs" module in Node.js?
20. Describe the role of the "path" module in working with file paths.

**HTTP and Networking:**

21. What is the "http" module in Node.js, and how can you create an HTTP server?
22. Explain how to make HTTP requests using the "http" or "https" module in Node.js.
23. How do you handle routing and request/response handling in an HTTP server in Node.js?
24. Describe the purpose of the "url" and "querystring" modules in Node.js.
25. What is the "net" module, and how is it used for low-level network communication in Node.js?

**Event Emitters and Streams:**

26. What are event emitters in Node.js, and how can you create custom ones?

27. Explain the concept of streams in Node.js and their use cases.
28. How can you create readable and writable streams in Node.js?
29. Describe the "transform" stream and its role in data transformation.
30. What is the "pipe" method, and how is it used for stream processing?

**Working with Data:**

31. How can you work with JSON data in Node.js?
32. Explain the purpose of the "buffer" module and how it handles binary data.
33. How do you perform database operations in Node.js, including CRUD operations?
34. What is a NoSQL database, and how does it compare to traditional relational databases?
35. How can you use third-party libraries like Mongoose for database operations in Node.js?

**Error Handling and Debugging:**

36. How do you handle errors and exceptions in Node.js?
37. Explain the use of try-catch blocks for error handling in asynchronous code.
38. What is the "util" module, and how does it enhance error handling and debugging?
39. How can you enable debugging and inspect code using Node.js's built-in debugging tools?
40. Describe best practices for error logging and monitoring in Node.js applications.

**Node Package Manager (NPM):**

41. What is NPM, and how is it used to manage Node.js packages and dependencies?
42. How do you install and update packages using NPM?
43. Explain the purpose of "package.json" and its key sections.
44. How can you publish your own Node.js package to the NPM registry?
45. What is the "npm audit" command, and how is it used for security vulnerability checks?

**Performance Optimization:**

46. What strategies and techniques can be employed to optimize the performance of Node.js applications?
47. Describe the use of clustering to take advantage of multi-core processors in Node.js.
48. How can you profile and debug memory leaks in Node.js applications?
49. Explain the role of load balancing and reverse proxy servers in scaling Node.js applications.
50. What is the "event-driven, non-blocking I/O" nature of Node.js, and how does it contribute to its performance?